```
************************************************************************
*          Molecular Dynamics code to simulate at NVE collectivity
*          a system of N atoms of Ar inside a cubic box, interacting
*          through a truncated Lennard-Jones force field at 2.5 sigma.
*          Leap-frog Verlet integration algorithm.
*
************************************************************************
      PROGRAM leapfroglj
      implicit double precision(a-h,o-z)
      double precision mass

c     1. Defining dimensions

      dimension r(3,1000),vinf(3,1000),accel(3,1000)

c     2. Reading data and computing related quantities

      open(1,file='leap-lj.data',status='old')
      read(1,*) nconf
      read(1,*) natoms
      read(1,*) mass
      read(1,*) sigma,epsil
      read(1,*) deltat
      close(1)

      nf = 3*natoms-3 !number of degrees of freedom
      rc = 2.5d0 !sigma = range of the potential in reduced units

c     3. Reading initial configuration (positions, velocities) in A and A/ps

      open(2,file='leap-conf.data',status='old')
      do is = 1,natoms
         read(2,*) (r(l,is),l=1,3)
         read(2,*) (vinf(l,is),l=1,3)
      end do
      read(2,*) boxlength
      close(2)
c
c     Opening files to write results
c
      open(3,file='energy-leap.dat',status='unknown')
      open(4,file='temp-leap.dat',status='unknown')
c
c     4. Change to reduced units

      call reduced(natoms,r,vinf,boxlength,deltat,epsil,sigma,
     &mass,uvel)

c     5. Start the loop to generate new configurations

      do i = 1,nconf
         call forces(natoms,r,boxlength,accel,rc,epot)
         call velpos(natoms,vinf,accel,deltat,r,nf,ecin,temp,
     &               boxlength)
         etot = ecin + epot
         write(3,*) i*deltat, etot
         write(4,*) i*deltat, temp
      enddo
      close(3)
      close(4)

c     6. Saving last configuration in A and A/ps

      open(11,file='newconf.data',status='unknown')
      do is = 1,natoms
         write(11,*) (r(l,is)*sigma,l=1,3)
```

mcnvt.f

file='mc-lj.data'

ncycl, nsamp

tref

deltax

T*=0.7, rho*=0.83 (aprox.)

do icycl=1,ncycl

call mcmove(.....)

if (mod(icycl,nsamp).eq.0) then
  call sample(....)
end if

```
         write(11,*) (vinf(l,is)*uvel,l=1,3)
      end do
      write(11,*) boxlength*sigma
      close(11)

      stop
      end

************************************************************
************************************************************
c              subroutine reduced
************************************************************
************************************************************

      subroutine reduced(natoms,r,vinf,boxlength,deltat,
     &epsil,sigma,mass,uvel)
      implicit double precision(a-h,o-z)
      double precision mass
      dimension r(3,1000),vinf(3,1000)

      rgas = 8.314472673d0 !J/(mol*K)
      utime = sigma*dsqrt(mass/epsil)*dsqrt(10.d0/rgas)
      uvel = sigma/utime !unit of velocity, expressed in A/ps

      boxlength = boxlength/sigma
      deltat = deltat/utime
      do is = 1,natoms
         do l = 1,3
            r(l,is) = r(l,is)/sigma
            vinf(l,is) = vinf(l,is)/uvel
         end do
      end do

      return
      end

************************************************************
************************************************************
c              subroutine forces
************************************************************
************************************************************

      subroutine forces(natoms,r,boxlength,accel,rc,epot)
      implicit double precision(a-h,o-z)
      dimension r(3,1000),accel(3,1000)

      do is = 1,natoms
         do l = 1,3
            accel(l,is) = 0.d0 !sets accelerations to 0
         end do
      end do
      epot = 0.d0

c      atom-atom interactions

      do is = 1,natoms-1
         do js = is+1,natoms
            call lj(is,js,r,boxlength,accel,rc,pot)
            epot = epot + pot
         end do
      end do

      return
      end

************************************************************
```

It must be simplified
we don't have velocities!

It must be simplified (we don't need the accelerations, just the energy).

```
************************************************************
c              subroutine Lennard-Jones
************************************************************
************************************************************

      subroutine lj(is,js,r,boxlength,accel,rc,pot)
      implicit double precision(a-h,o-z)
      dimension r(3,1000),accel(3,1000)
      dimension rij(3)

      rr2 = 0.d0
      pot = 0.d0
      do l = 1,3
         rijl = r(l,js) - r(l,is)
         rij(l) = rijl - boxlength*dnint(rijl/boxlength)
         rr2 = rr2 + rij(l)*rij(l)
      end do

      rr = dsqrt(rr2)

      if (rr.lt.rc) then
         ynvrr2 = 1.d0/rr2
         ynvrr6 = ynvrr2*ynvrr2*ynvrr2
         ynvrr12 = ynvrr6*ynvrr6
         forcedist = 24.d0*(2.d0*ynvrr12-ynvrr6)*ynvrr2
         pot = 4.d0*(ynvrr12-ynvrr6)
         do l = 1,3
            accel(l,is) = accel(l,is) - forcedist*rij(l)
            accel(l,js) = accel(l,js) + forcedist*rij(l)
         end do
      end if

      return
      end

************************************************************
************************************************************
c              subroutine velpos
************************************************************
************************************************************

c       Calculating velocity at instants t+delta/2 and t
c       and getting temporal position at instant t + deltat.

      subroutine velpos(natoms,vinf,accel,deltat,r,nf,ecin,temp,
     &                  boxlength)

      implicit double precision(a-h,o-z)
      dimension vinf(3,1000),accel(3,1000),r(3,1000)

      ecin = 0.d0
      do is = 1,natoms
         v2 = 0.d0
         do l = 1,3
            vsup = vinf(l,is) + accel(l,is)*deltat
            r(l,is) = r(l,is) + vsup*deltat
            v = (vsup+vinf(l,is))/2.d0
            v2 = v2 + v*v
            vinf(l,is) = vsup
         end do
         ecin = ecin + 0.5d0*v2
      end do
      temp = 2.d0*ecin/dfloat(nf)
ccccc
ccccc       Applying  periodic boundary conditions
ccccc
```

It must be simplified (we don't need the accelerations, just the energy).

Replace it by mcmove(.....)

No

```
       Do is=1,natoms
          do l=1,3                                                    Yes
             if (r(l,is).lt.0) r(l,is) = r(l,is) + boxlength
             if (r(l,is).gt.boxlength) r(l,is) = r(l,is) - boxlength
          end do
      end do
ccccc
      return
      end
```